SYSTEM DESIGN

Caching Strategies Visual Guide

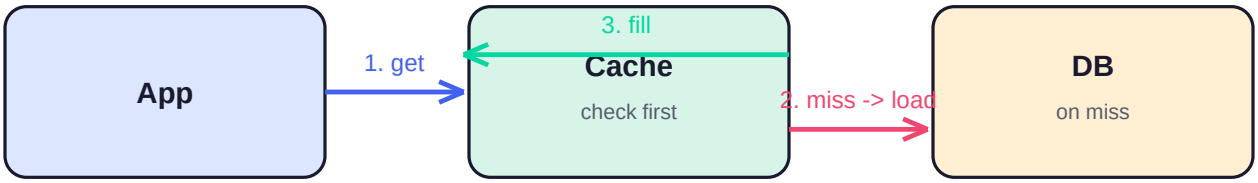
Trade memory for speed:
read/write paths, TTLs, eviction, and stampedes.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Cache-Aside (Lazy Loading)

The app owns the logic; the cache is just a fast shelf.

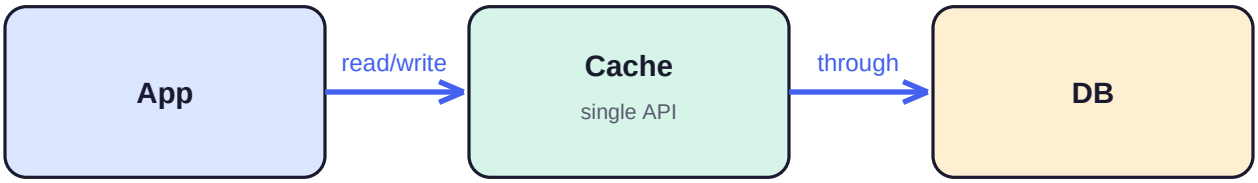


- Hit: serve from cache. Miss: load from DB, write to cache, then serve.
- Writes go straight to the DB and invalidate (or update) the cache entry.
- Pros: cache holds only hot data; a dead cache never blocks reads. Cons: first read is slow, stale window on write.

STEP 2/6

Read/Write-Through

The cache sits IN the path and owns the data flow.



- Read-through: cache loads from DB on miss (app code stays simple).
- Write-through: every write lands in cache AND DB before ack -- always consistent, write latency doubles.
- Best when the same data is read and written through one layer (ORM-style caches).

STEP 3/6

Write-Back (Write-Behind)

Acknowledge fast, persist lazily. Speed with a risk window.

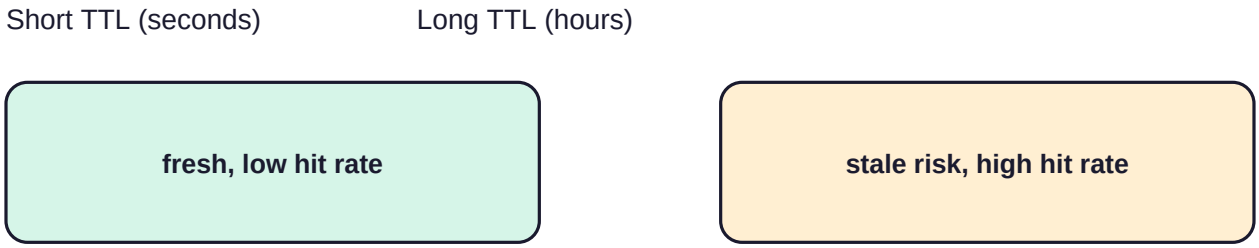
Crash before flush = data loss. Use only where loss is tolerable or the log is durable (write-ahead log).

- Wins: absorbs write bursts, coalesces repeated writes to the same key.
- CPUs do this (L1/L2 caches); databases do NOT for user data -- durability first.

STEP 4/6

TTL: Freshness on a Timer

Every entry carries an expiry; staleness becomes a knob.

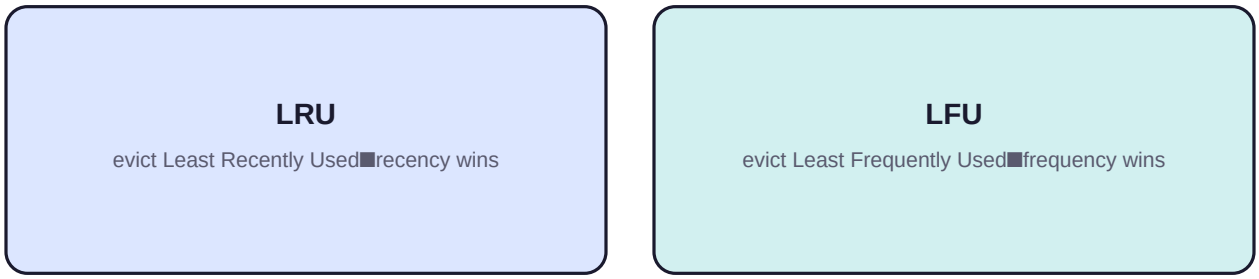


- TTL bounds staleness: the worst case is 'data as of TTL seconds ago' -- a real SLA.
- Jitter TTLs (+/- 10%) so a million keys do not expire in the same second.
- Event-driven invalidation (publish 'user:42 changed') beats short TTLs for correctness-critical data.

STEP 5/6

Eviction: LRU and LFU

Memory is finite; the policy picks the victim.

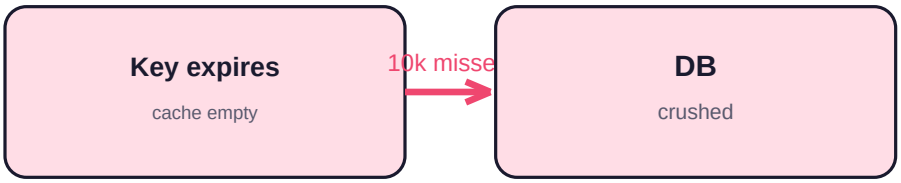


- LRU: great for recency workloads (sessions, feeds). Weakness: a full scan poisons the cache.
- LFU: great for popularity skew (viral content). Weakness: stale favorites never leave (needs aging).
- Redis: volatile-lru / allkeys-lfu / TinyLFU admission -- pick by workload, measure hit ratio.

STEP 6/6

Thundering Herd (Dogpile)

A hot key expires; ten thousand requests hit the DB at once.



Fix: single-flight

1 loads, rest wait

Fix: stale-while-revalidate

serve old, refresh async

Probabilistic early refresh (x-fetch): recompute BEFORE expiry with p rising as TTL nears.

STEP ✓/✓

Key Takeaways

Caching Strategies — Remember

- Cache-aside: app checks cache, loads on miss -- simple, cache holds only hot data.
- Write-through keeps cache and DB in lockstep; write-back is fast but risks loss on crash.
- TTL turns staleness into a tunable SLA; jitter expiries to avoid synchronized stampedes.
- LRU favors recency, LFU favors popularity -- match the policy to the workload.
- Defeat thundering herds with single-flight, stale-while-revalidate, or early refresh.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev